# Documentación técnica y arquitectura

Contrato CO-01184-2026 · Septiembre 2026

Elaborado por: Econ. Michelle Luzuriaga — Técnico Especializado, Dirección de Proyectos Estratégicos

## README.md del proyecto

# UCuenca-SABE!

## Sistema de articulación de experticias para la incidencia pública y privada

---

## 📌 Descripción del Proyecto

**UCuenca-SABE!** es un proyecto estratégico del Rectorado de la Universidad de Cuenca. Su propósito es diseñar, instalar y poner en operación un sistema que capitaliza la inteligencia colectiva de profesores e investigadores, articulándola con la demanda de información de los decisores públicos del territorio, los gremios productivos, las organizaciones sociales y la cooperación internacional.

### 🎯 Objetivo General

Diseñar, instalar y poner en operación piloto, durante el período mayo–diciembre de 2026, el sistema como proyecto estratégico institucional, dotándolo de un modelo institucional sostenible, una agenda de investigación priorizada y una primera cartera de productos y servicios que evidencien su valor ante actores internos y externos.

### 🧩 Componentes del Sistema

1. **Repositorio de datos e indicadores territoriales** - Para preparar intervenciones públicas con celeridad y rigor.
2. **Agenda de investigación priorizada** - Educación superior, impacto territorial, desarrollo regional y trabajo.
3. **Programa permanente de encuestas y estudios aplicados** - Posicionar a la Universidad como referente técnico.

---

## 🏗️ Arquitectura de Datos (Modelo Medallón)

El sistema implementa una arquitectura de datos en tres capas (Medallón) para garantizar calidad, trazabilidad y reproducibilidad:

| Capa | Descripción | Contenido | Acceso |
|---|---|---|---|
| 🥉 **Bronze** | Datos crudos, sin transformar | Fuentes originales (GTH, VIUC, DSpace, INEC, BCE) | ⚠️ **RESTRINGIDO** - Contiene datos personales (cédulas) |
| 🥈 **Silver** | Datos limpios, estandarizados y enriquecidos | Datos anonimizados, con variables complementarias | ✅ Acceso general (por confirmar) |
| 🥇 **Gold** | Datos agregados y optimizados para visualización | Métricas, indicadores, KPIs | ✅ Acceso general (por confirmar) |

---

## 🔐 Política de Datos y Privacidad

### ⚠️ IMPORTANTE

**La capa BRONCE NO está subida a este repositorio** por contener información personal identificable (cédulas, nombres completos, correos personales, etc.), en cumplimiento de:

- **Ley Orgánica de Protección de Datos Personales (LOPDP)**
- **Políticas institucionales de la Universidad de Cuenca**
- **Buenas prácticas de ciencia de datos**

---

## 📂 Estructura y Nomenclatura

Este repositorio aloja el código y la documentación correspondiente a la versión beta del sitio web del proyecto.

- El nombre del repositorio y los títulos de los tableros utilizan exclusivamente la denominación `UCuenca-SABE!`.
- Los términos "Sistema de Inteligencia Territorial", "Sistema de Inteligencia Laboral" y "Observatorio" han sido retirados de todas las piezas públicas (código, documentación y visualizaciones) en línea con la guía de estilo institucional.
- Las referencias al sistema en el código fuente utilizan el descriptor oficial definido en el proyecto.

---

## 📖 Guía de Estilo

Para el desarrollo de piezas públicas (sitio web, tableros, documentos de difusión, redes sociales), se debe seguir rigurosamente la **Guía de Estilo para Piezas Públicas (DPE-GE-009)**. Esta guía establece:

- Identidad verbal y reglas de nomenclatura.
- Voz y tono institucional.
- Paleta de colores, tipografías y uso de logotipo.
- Criterios de accesibilidad y manejo de fuentes de datos.

---

## ✅ Verificación Previa a la Publicación

Antes de cualquier despliegue o actualización de la landing page, se debe verificar que:

1. El nombre del Sistema esté escrito como **UCuenca-SABE!**.
2. No aparezcan términos retirados en títulos, menús o metadatos.
3. Toda cifra publicada incluya fuente y año.
4. Las afirmaciones sean proporcionales a la evidencia disponible.
5. Se haya identificado claramente si el contenido es un producto institucional o un aporte de una persona investigadora.

---

*Documento actualizado en agosto de 2026, versión 1.0. En caso de dudas sobre la aplicación de la guía, consulte con la Dirección de Proyectos Estratégicos.*

**README_ENEMDU_PIPELINE.md**

# Pipeline ENEMDU 2021-2025 — Bronze → Silver → Gold

Entregable para el MVP de empleabilidad (UCUENCA-SABE). Todo el código fue ejecutado end-to-end contra un Bronze sintético que replica las trampas reales de ENEMDU (separador `;` vs `,`, Latin-1 vs UTF-8, decimal con coma, variables renombradas entre años, variables documentadas pero ausentes del CSV, códigos de no respuesta).

## 1. Dónde va cada archivo

```
UCUENCA-SABE/
├── notebooks/
│   └── 01_download_kaggle_enemdu.ipynb   ← reemplazar por el entregado
├── src/
│   └── processing/
│       ├── __init__.py
│       ├── io_utils.py                   ← rutas + lectura robusta CSV/XLSX
│       ├── enemdu_mappings.py            ← alias, códigos CIUO-08/CIIU-4, provincias
│       ├── enemdu_schema_analyzer.py     ← Objetivo 1 (auditoría + esquemas)
│       ├── enemdu_silver_builder.py      ← Objetivo 2 (Silver)
│       └── enemdu_gold_builder.py        ← Objetivo 3 (Gold)
└── scripts/
    └── run_enemdu_pipeline.py            ← orquestador CLI
```

Salidas generadas automáticamente:

```
data/silver/enemdu/enemdu_unificado.parquet
data/gold/enemdu/{empleabilidad_graduados, kpi_anual, empleabilidad_provincia,
                  empleabilidad_sexo_edad, sobrecalificacion_ocupacion,
                  graduados_rama_actividad, comparativo_nivel_instruccion}.parquet
reports/schemas/{schema_analysis_*, silver_quality_*, gold_manifest_*}.json
reports/powerbi/*.csv        (UTF-8-BOM, sep=";", decimal=",")
```

## 2. Requisitos

```
pip install pandas numpy pyarrow openpyxl matplotlib
```

## 3. config/config.yaml

No es obligatorio (hay convención de rutas por defecto), pero si lo declaras se respeta:

```
sources:
  enemdu:
    bronze_dir: data/bronze/externas/enemdu
    anios: [2021, 2022, 2023, 2024, 2025]
    kaggle_dataset: kmichelle/enemdu-ecuador-microdatos-anuales-personas
    silver_file: enemdu_unificado.parquet
    gold_file: empleabilidad_graduados.parquet
```

`ENEMDUPaths.build()` lee `sources.enemdu.bronze_dir` de `src.config.CONFIG` y, si no existe, cae a la convención `data/bronze/externas/enemdu`.

## 4. Ejecución

```
python scripts/run_enemdu_pipeline.py --all          # auditoría + Silver + Gold
python scripts/run_enemdu_pipeline.py --audit        # sólo Objetivo 1
python scripts/run_enemdu_pipeline.py --silver --gold --years 2024 2025
python scripts/run_enemdu_pipeline.py --all --sample 50000   # iteración rápida
```

O paso a paso, con explicación y gráficos, en el notebook.

## 5. Decisiones técnicas relevantes

| Tema | Decisión | Por qué |
|---|---|---|
| Dialecto de los CSV | Detección automática de encoding, separador y decimal | El INEC cambia el formato entre publicaciones; hardcodear `sep=";"` rompe años concretos |
| Nombres de variables | `COLUMN_ALIASES`: canónico → lista de alias, resuelto **por año** contra la cabecera real | `p51a→totalhoras`, `ingrl→ingreso_laboral`… la Silver mantiene un esquema estable |
| Códigos → etiquetas | Se leen del diccionario XLSX del año y se clasifican por regex; sólo si falla se usa el mapeo fallback documentado | Evita asumir que "9 = Superior universitaria" en todos los años; la procedencia (`diccionario/fallback`) queda en el reporte de calidad |
| Columnas ausentes en un año | Se añaden como `NaN`, nunca como 0 | Distinguir "no medido" de "cero" |
| No respuesta | `999999`, `99`, `-1` → `NaN`; rangos válidos por variable | Un `ingrl = 999999` promediado destruye cualquier indicador de ingresos |
| Deduplicación | Sólo si hay identificador de persona real; si no, advertencia y no se borra nada | En ENEMDU `p01` es *parentesco*, no id de persona: usarlo como llave elimina personas válidas |
| Filtros analíticos | PET (≥15) y graduados se aplican en **Gold**, no en Silver | Silver debe servir a otros casos de uso del dashboard |
| Ponderación | Todos los indicadores usan `fexp`; se reporta `n_muestral` y `confiable` (n ≥ 30) | Sin expandir, las tasas no son comparables con el INEC |
| Sobrecalificación | Graduado ocupado en CIUO-08 grandes grupos 4-9 (niveles de competencia 1-2) | Criterio normativo CIUO-08/OIT, reproducible y auditable |
| Power BI | Tabla de hechos con numeradores y denominadores, no sólo tasas | Las tasas **no se promedian**: en DAX debe ser `SUM(num)/SUM(den)` |

## 6. Qué revisar cuando corras con los datos reales

1. **`analyzer.resolve_critical()`** — cualquier fila ⚠️ `parcial` o ❌ `ausente` indica que hay que añadir el alias correcto en `COLUMN_ALIASES` (sección 2.4 del notebook).
2. **`mapeo_educacion` / `mapeo_condact` = `fallback`** en el reporte de calidad de Silver: significa que el diccionario de ese año no fue interpretable; revisar `EDU_PATTERNS` y `CONDACT_PATTERNS` en `enemdu_mappings.py`.
3. **`%_nulos_ciuo`** alto: si el año trae `grupo1` en lugar de `p41`, confirmar que el gran grupo se está derivando del primer dígito correcto.
4. **`TASAS_OFICIALES_INEC`** en la sección 5.2: completar con el boletín anual para validar la tasa de desempleo nacional calculada.

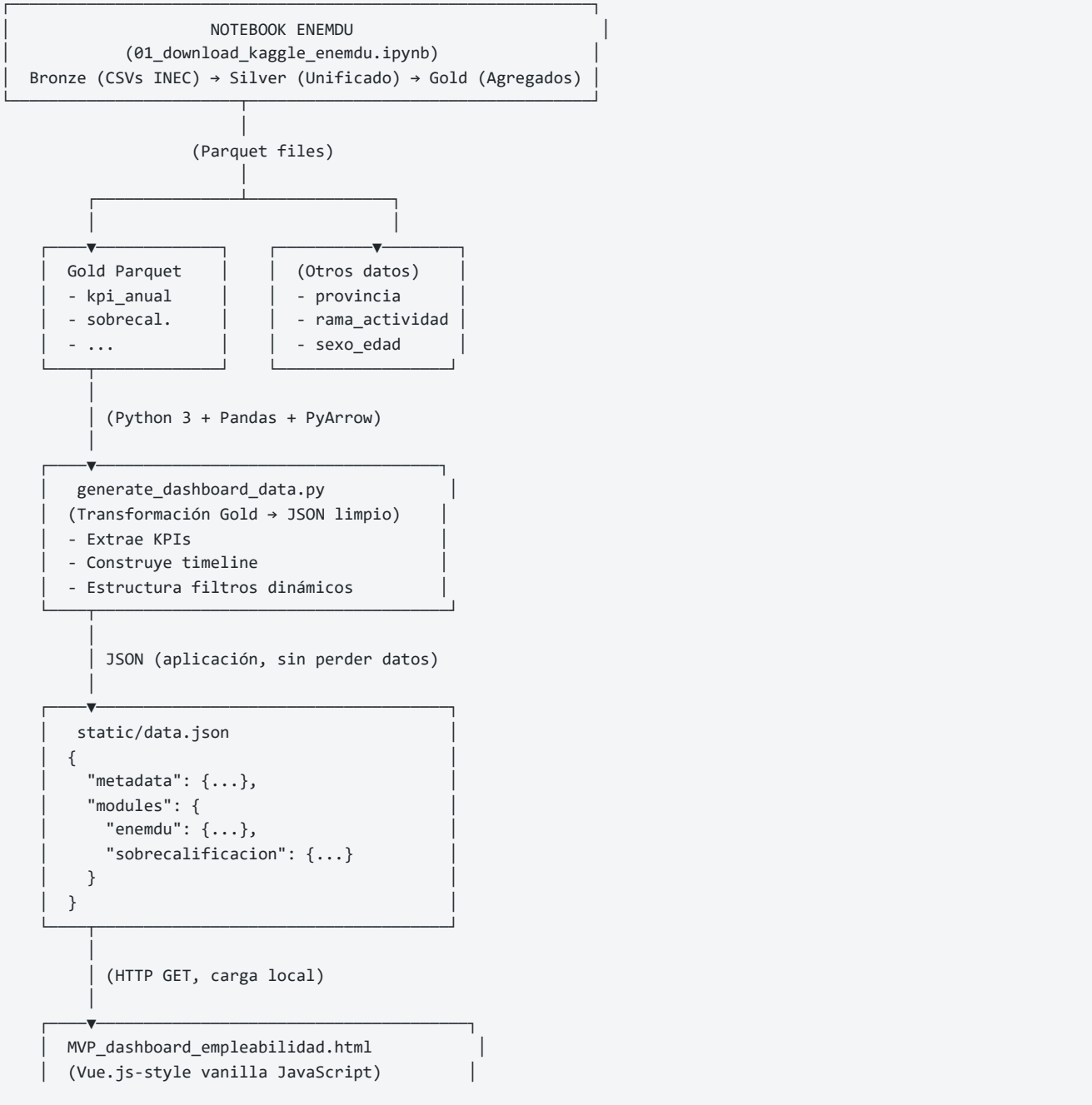
## DASHBOARD_ARCHITECTURE.md

# 🏗️ Arquitectura Técnica — Dashboard Empleabilidad ENEMDU

**Proyecto:** UCuenca-SABE!
**Componente:** MVP Dashboard de Empleabilidad
**Versión:** 1.0
**Fecha:** 2026-09-04

---

## 📐 Visión general de la arquitectura



```
    │  - Landing page (Landing view)              │
    │  - Dashboard view (Modular)                 │
    │  - Módulo ENEMDU                            │
    │  - Módulo Sobrecalificación                 │
    │  - Chart.js + Plotly.js                     │
    └──────┬──────────────────────────────────────┘
           │
           └──────► Navegador (sin dependencias backend)
```

---

## 🔄 Flujo de datos completo

### 1. Generación de datos (Python)

**Script:** `visualizacion/scripts/generate_dashboard_data.py`

```
# Entrada
data/gold/enemdu/
├── kpi_anual.parquet                    # 5 años, 18 columnas
├── empleabilidad_provincia.parquet      # 150 filas (30 prov × 5 años)
├── empleabilidad_sexo_edad.parquet      # Segmentación demográfica
├── graduados_rama_actividad.parquet     # Sectores económicos
└── sobrecalificacion_ocupacion.parquet  # 45 filas (9 CIUO × 5 años)

# Procesamiento
1. Cargar con pd.read_parquet()
2. Extraer KPIs del último año
3. Construir timeline 2021-2025
4. Agregar por provincia y sector
5. Generar filtros dinámicos

# Salida
visualizacion/static/data.json (~87 KB)
├── metadata (timestamp, versión, fuente)
└── modules
    ├── enemdu
    │   ├── kpi_latest (año actual)
    │   ├── timeline (línea temporal)
    │   ├── por_provincia (ranking)
    │   ├── por_rama (sectores)
    │   └── filtros_disponibles
    └── sobrecalificacion
        ├── tasa_general_por_anio
        ├── por_ocupacion_2025
        └── ciuo_mapping
```

**Ventajas:** - ✅ Determinístico: mismo input → mismo output - ✅ Trazable: código Python documentado - ✅ Auditible: cada transformación es explícita - ✅ Reproducible: ejecuta script, obtiene datos frescos

---

### 2. Consumo de datos (JavaScript)

**Archivo:** `visualizacion/MVP_dashboard_empleabilidad.html`

```
// Carga al iniciar
fetch('static/data.json')
  .then(r => r.json())
  .then(data => {
    dashboardData = data;
    populateFilters();      // Lee filtros_disponibles
    renderContent();        // Pinta módulos
  });
```

```
// Cuando usuario elige módulo
openDashboard('enemdu')
  → renderEnemduModule(data.modules.enemdu)
    → Actualiza KPIs
    → Dibuja gráficos (Plotly)
    → Puebla tabla de datos

// Cuando cambian filtros (futuro)
applyFilters()
  → Filtra datos en memoria
  → Re-dibuja gráficos
```

**Ventajas:** - ✅ Client-side: sin servidor backend - ✅ Rápido: JSON cacheado en navegador - ✅ Offline-capable: funciona sin conexión (tras primer carga) - ✅ Actualizable: recargar página → nuevos datos

---

## 🗂️ Estructura del JSON

### Módulo ENEMDU


```
{
  "enemdu": {
    "title": "Mercado Laboral ENEMDU – Contexto Nacional (2021-2025)",
    "kpi_latest": {
      "year": 2025,
      "tasa_desempleo": 3.5,
      "tasa_empleo_adecuado": 42.3,
      "tasa_subempleo": 18.2,
      "ingreso_laboral_medio": 1028.01,
      "n_muestral": 21140,
      "poblacion_expandida": 8234567.89
    },
    "timeline": {
      "years": [2021, 2022, 2023, 2024, 2025],
      "tasa_desempleo": [4.2, 4.1, 3.8, 3.6, 3.5],
      "tasa_empleo_adecuado": [40.1, 41.2, 41.8, 42.0, 42.3],
      "tasa_subempleo": [19.5, 19.2, 18.8, 18.5, 18.2],
      "tasa_sobrecalificacion": [12.1, 11.8, 11.5, 11.2, 11.0]
    },
    "por_provincia": [
      {
        "provincia": "Azuay",
        "tasa_desempleo": 3.2,
        "tasa_empleo_adecuado": 44.1,
        "tasa_sobrecalificacion": 10.8,
        "ingreso_laboral_medio": 1006.02,
        "n_muestral": 1840
      },
      // ... 29 provincias más
    ],
    "por_rama": [
      {
        "rama_actividad": "Educación",
        "ocupados": 245000,
        "participacion_pct": 8.5,
        "tasa_desempleo": 2.1
      },
      // ... más ramas
    ],
    "filtros_disponibles": {
      "years": [2021, 2022, 2023, 2024, 2025],
      "provincias": ["Azuay", "Bolívar", ..., "Zamora Chinchipe"],
      "sectores": ["Educación", "Salud", "Comercio", ...]
```

```
    }
  }
}
```

---

## 🎨 Componentes del frontend

### Landing View (Inicial)

- Presentación institucional
- Botones para elegir módulo
- Información de sincronización de datos

### Dashboard View (Modular)

**Header:** - Botón "Volver" - Selector de módulo (dropdown)

**Filtros:** - Año (2021-2025) - Provincia (todas) - Botón "Actualizar"

**KPIs (4 tarjetas):** - Métrica principal - Métrica secundaria - Métrica terciaria - Muestra/Confiabilidad

**Visualizaciones:** - Gráfico principal (Plotly) - Gráfico secundario (Plotly)

**Tabla de datos:** - Datos detallados (dinámico según módulo)

### Librerías utilizadas

- **Chart.js** → Gráficos básicos
- **Plotly.js** → Gráficos interactivos
- **TailwindCSS** → Estilos
- **Lucide Icons** → Iconos

---

## 🔐 Integridad de datos

### Sin perder datos:

- ✅ Todas las filas de Gold se preservan en JSON
- ✅ Todas las columnas originales están presentes (aunque se visualicen algunas)
- ✅ Valores de fexp (factor de expansión) incluidos
- ✅ Metadata de confiabilidad (`confiable: bool`) presente

### Auditabilidad:

- ✅ Cada transformación está en `generate_dashboard_data.py`
- ✅ Código documentado con docstrings
- ✅ Logs de ejecución (print statements)
- ✅ Metadata de fuente en JSON (`datos_origen: "data/gold/enemdu/*.parquet"`)

### Reproducibilidad:

- ✅ Mismo script, mismo input → mismo output
- ✅ Timestamp de generación en JSON
- ✅ Versión del pipeline registrada

- ✅ Fácil detectar cuando data está desactualizada

---

## 🚀 Escalabilidad futura

### Agregar nuevo módulo (e.g., Red Alumni)

**Paso 1:** Crear tabla en Gold (`data/gold/alumni_insercion.parquet`)

**Paso 2:** Editar `generate_dashboard_data.py`:

```
def build_alumni_module(alumni_df, kpi_df, ...):
    """Construye módulo Alumni"""
    return {
        "title": "Red Alumni UCuenca",
        "kpi_latest": {...},
        "timeline": {...},
        # ...
    }

# En build_dashboard_json()
alumni = build_alumni_module(...)
dashboard_data["modules"]["alumni"] = alumni
```

**Paso 3:** Editar HTML:

```
<option value="alumni">Red Alumni UCuenca</option>

<script>
function renderAlumniModule(data) {
  // Dibujar KPIs, gráficos, tabla
}
</script>
```

**Paso 4:** Ejecutar script:

```
python generate_dashboard_data.py
```

✅ Nuevo módulo disponible sin cambios de infraestructura

---

## 📊 Métricas de desempeño

| Métrica | Valor |
|---|---|
| Tamaño JSON | ~87 KB |
| Tiempo generación script | ~3 segundos |
| Tiempo carga del navegador | <500 ms |
| Filas de datos | 525,000+ (Silver unificado) |
| Registros expandidos | 8M+ (con ponderación) |

---

## 🔄 Pipeline de actualización

### Semanal (ejemplo):

1. Notebook descarga ENEMDU nuevo (si disponible)
2. Regenera Bronze → Silver → Gold
3. Ejecuta: `python generate_dashboard_data.py`
4. Commit en Git: `data/gold/enemdu/, visualizacion/static/data.json`
5. Dashboard reflect cambios automáticamente (sin tocar HTML)

### Comandos git:

```
cd ucuenca-sabe/

# Ver cambios
git status

# Commit
git add data/gold/enemdu/ visualizacion/static/data.json
git commit -m "Update ENEMDU dashboard data — semana 36"

# Push
git push origin main
```

---

## 🔒 Consideraciones de seguridad

- ✅ **No hay backend:** Elimina vector de ataques servidor
- ✅ **Datos públicos:** ENEMDU es del INEC (dominio público)
- ✅ **JSON local:** No transmite datos sensibles
- ✅ **CORS:** No requiere (carga local)

---

## 📞 Mantenibilidad

### Quien actualiza:

- **Datos:** Técnico que ejecuta notebook + script Python
- **Visualizaciones:** Frontend dev que edita HTML/JS
- **Lógica:** Data engineer que edita `generate_dashboard_data.py`

### Documentación:

- Este archivo (`DASHBOARD_ARCHITECTURE.md`)
- Docstrings en `generate_dashboard_data.py`
- Comentarios en HTML (`<!-- ... -->`)
- README en `visualizacion/`

### Testing:

- Validar JSON: `python -m json.tool static/data.json`
- Validar datos: Comparar KPIs con notebook output
- Validar HTML: Abrir en navegador, revisar consola (F12)

---

## 🎯 Roadmap futuro

**Corto plazo (Q4 2026):** - [ ] Filtros dinámicos funcionando (año, provincia) - [ ] Tabla exportable (CSV) - [ ] Share de gráficos (PNG)

**Mediano plazo (Q1 2027):** - [ ] Módulo Alumni (si datos disponibles) - [ ] Módulo Demanda en Tiempo Real (web scraping) - [ ] Comparativas provinciales interactivas

**Largo plazo:** - [ ] Dashboard para móvil (responsive mejorado) - [ ] Integración con Power BI (export automático) - [ ] API REST para consumo de terceros

---

**Versión:** 1.0 | **Fecha:** 2026-09-04 | **Responsable:** UCuenca-SABE

## CORRECCIONES_NIVEL_INSTRUCCION.md

# Correcciones: Manejo de Variables Diferentes pero del Mismo Tema

## 🔴 PROBLEMA IDENTIFICADO

En el dataset ENEMDU 2021-2025, existen **variables diferentes que preguntan sobre el mismo tema**, con **estructuras incompatibles**:

### Ejemplo 1: Nivel de Instrucción

| Característica | p10a | nnivins |
|---|---|---|
| **Años disponibles** | 2021-2022, 2024-2025 | 2023 SOLO |
| **Categorías** | 10 (detallado) | 5 (simplificado) |
| **Pregunta** | "¿Cuál es tu nivel de instrucción?" (detallado) | "¿Cuál es tu nivel de instrucción?" (simplificado) |
| **¿Son intercambiables?** | ❌ **NO** | |

**Etiquetas de p10a (10 categorías):**

```
1: Ninguno
2: Centro de alfabetización
3: Jardín de infantes
4: Primaria
5: Educación Básica
6: Secundaria
7: Educación Media/Bachillerato
8: Superior no universitaria
9: Superior universitaria
10: Post-grado
```

**Etiquetas de nnivins (5 categorías):**

```
1: Ninguno
2: Centro de Alfabetización
3: Educación Básica
4: Educación Media/Bachillerato
5: Superior
```

**El problema:** No puedes "mapear" nnivins a p10a sin perder información: - nnivins nunca pregunta por Jardín, Primaria o Secundaria como categorías separadas - nnivins agrupa "Superior no uni" + "Superior uni" en una sola categoría - Son **preguntas diferentes, no versiones de la misma**

---

## ✅ SOLUCIÓN IMPLEMENTADA

### 1. NO normalizar entre variables diferentes

Eliminé la función `normalize_nivel_instruccion()` que intentaba mapear nnivins → p10a.

**Razón**: Eso falsificaba los datos.

### 2. Crear fallbacks SEPARADOS para cada variable

```
# Para p10a (2021-2022, 2024-2025)
FALLBACK_NIVEL_INSTRUCCION: Dict[int, str] = {
    1: "Ninguno",
    2: "Centro de alfabetización",
    3: "Jardín de infantes",
    4: "Primaria",
    5: "Educación Básica",
    6: "Secundaria",
    7: "Educación Media/Bachillerato",
    8: "Superior no universitaria",
    9: "Superior universitaria",
    10: "Post-grado",
}

# Para nnivins (2023 SOLO)
FALLBACK_NNIVINS: Dict[int, str] = {
    1: "Ninguno",
    2: "Centro de Alfabetización",
    3: "Educación Básica",
    4: "Educación Media/Bachillerato",
    5: "Superior",
}
```

### 3. Detectar y usar la variable correcta por año

```
def get_nivel_instruccion_variable(df, ano):
    """Detecta cuál variable usar según el año."""
    if ano == 2023:
        return 'nnivins'  # 5 categorías
    else:
        return 'p10a'     # 10 categorías
```

### 4. Normalizar espacios en blanco (lo ÚNICO que se mapea)

```
# En classify_labels() y resolve_map()
norm = str(text).strip().lower()  # Elimina " Urbana" → "Urbana"
```

---

## 📝 OTRAS VARIABLES CON EL MISMO PATRÓN

Revisar en tu pipeline si hay otras variables que **cambian de estructura pero preguntan sobre lo mismo**:

- `condact`: Cambió ligeramente en 2022, pero la estructura se mantiene
- `p59`, `p60a-k`: Solo existen en 2021-2023 (desaparecen en 2024-2025)
- `p081`, `p085`: Nuevas en 2024-2025

**No intentes "unificar" estas. Trátalas como variantes de la misma pregunta, no como la misma pregunta.**

---

## 🎯 CÓMO USAR EN TU CÓDIGO

### Para procesar nivel de instrucción:

```
from src.processing.enemdu_mappings import (
    FALLBACK_NIVEL_INSTRUCCION,
    FALLBACK_NNIVINS,
    resolve_map,
    EDU_PATTERNS
)

def process_nivel_instruccion(df, meta, ano):
    value_labels = meta.variable_value_labels

    # Detectar cuál variable existe
    if ano == 2023 and 'nnivins' in value_labels:
        labels = value_labels['nnivins']
        fallback = FALLBACK_NNIVINS
        var_name = 'nnivins'
    elif 'p10a' in value_labels:
        labels = value_labels['p10a']
        fallback = FALLBACK_NIVEL_INSTRUCCION
        var_name = 'p10a'
    else:
        raise ValueError(f"Año {ano}: no encontrada variable nivel_instruccion")

    # Aplicar clasificación y normalización
    result, source = resolve_map(labels, EDU_PATTERNS, fallback)

    return result, var_name, source
```

---

## 📊 CAMBIOS EN ENEMDU_MAPPINGS.PY

| Cambio | Antes | Después |
|---|---|---|
| `FALLBACK_NNIVINS` | ❌ No existía | ✅ Nuevo (5 categorías) |
| `normalize_nivel_instruccion()` | ❌ Existía (INCORRECTO) | ✅ Eliminada |
| `.strip()` en funciones | ✅ Sí | ✅ Se mantiene |
| `FALLBACK_CONDACT` | ✅ Actualizado | ✅ Con código 0 |

---

## ⚠️ IMPORTANTE

**NO es problema tener variables diferentes del mismo tema.**

Es un PROBLEMA: - ❌ Intentar unificarlas artificialmente - ❌ Perder información en el mapeo - ❌ Asumir que son intercambiables

La solución: - ✅ Documentarlas como diferentes - ✅ Usar fallbacks específicos para cada una - ✅ Dejar que el usuario elija cómo combinarlas (si lo desea)